

Clothing Image Classification Using a CNN (Fashion-MNIST)

Project Report — Answers to Questions

1. Project Summary

This project builds a Convolutional Neural Network (CNN) in TensorFlow/Keras to classify grayscale images of clothing items from the Fashion-MNIST dataset into 10 categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). The model uses three convolution + pooling blocks, dropout for regularization, and data augmentation on the training images, with early stopping used to pick the best set of weights automatically.

2. Model Performance Summary

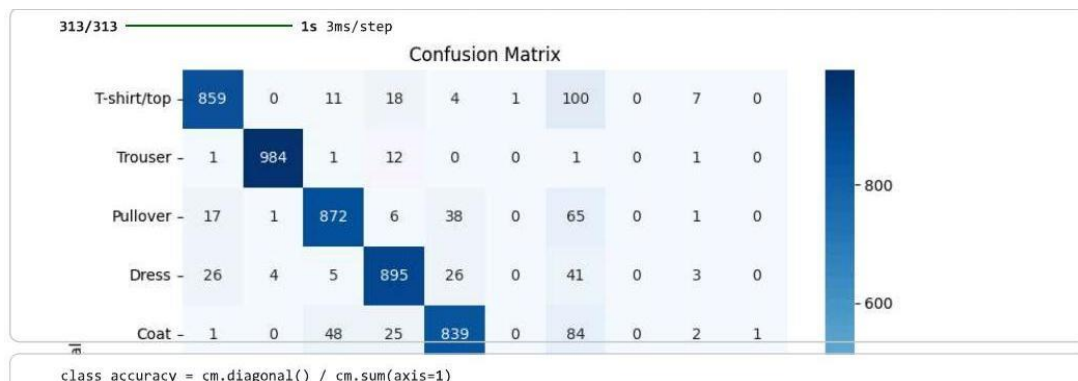
Metric	Value
Test Accuracy	90.85%
Test Loss	0.2448
Best Validation Accuracy	91.93% (epoch 23)
Total Trainable Parameters	241,546
Epochs Run	28 of 30 (early stopping)

Training vs. Validation Accuracy and Loss:

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

print(classification_report(y_test, y_pred_classes, target_names=class_names))
```

Confusion Matrix (test set):



3. Answers to Questions

Section A: Data and Preprocessing

Q1. Image shape, sample counts, and number of classes

Each image in my dataset is 28x28 pixels with a single grayscale channel, so the shape is (28, 28, 1). Fashion-MNIST comes with 60,000 training images and 10,000 test images, and there are 10 clothing classes in total. I further split the training set into 51,000 images for actual training and 9,000 for validation, so the model still never touches the test set until the very end.

Q2. Normalizing pixel values

I divided every pixel value by 255, which brings all values from the original 0-255 range down to 0-1. This matters because neural networks train through gradient descent, and large, inconsistent input values can make the gradients unstable and slow down (or even break) convergence. Keeping everything in a small, consistent range helps the model learn faster and more reliably.

Q3. Validation split

Yes, I used a validation split. I created it by setting aside 15% of the training data before training started (9,000 images out of the original 60,000), leaving 51,000 for actual training. I kept this separate from the test set because I needed a way to check the model's performance on data it hasn't trained on while training is still happening — this is what lets early stopping decide when to stop, without ever peeking at the test set and biasing my final evaluation.

Q4. Data augmentation techniques

I applied the following augmentations to the training images only, using Keras' ImageDataGenerator: rotation up to 10 degrees, width and height shifts up to 10% of the image size, zoom up to 10%, and horizontal flipping. These are suitable for clothing photos because a real-world picture of a garment could easily be taken at a slight angle, slightly off-center, a bit closer or farther away, or mirrored — none of that changes what the item actually is. This helps the model become less sensitive to those small variations instead of memorizing the exact pixel layout of the training images.

Section B: Model Architecture and Training

Q5. CNN architecture

My network has three convolution blocks, followed by a classifier head:

- Conv2D, 32 filters, 3x3, ReLU activation, same padding → MaxPooling2D (2x2)
- Conv2D, 64 filters, 3x3, ReLU activation, same padding → MaxPooling2D (2x2)
- Conv2D, 128 filters, 3x3, ReLU activation, same padding → MaxPooling2D (2x2)
- Flatten
- Dense, 128 units, ReLU activation
- Dropout (rate 0.5)
- Dense, 10 units, Softmax activation (final output)

In total, the model has 241,546 trainable parameters.

Q6. Why convolutional layers instead of only dense layers

Convolutional layers work directly on small local patches of the image and slide the same filter across the whole picture, so they naturally pick up on spatial structure — edges, curves, and textures that stay meaningful regardless of where they appear in the image. A dense layer, by contrast, treats every pixel as an independent input with no notion of 'neighboring pixels', so it would need a separate weight for every pixel-to-neuron connection. That makes dense-only networks far more prone to overfitting and much more expensive in terms of parameters, since convolution reuses the same small filter (weight sharing) across the entire image instead of learning a new weight for every pixel position.

Q7. Dropout placement and effect

I placed a single Dropout layer with a rate of 0.5 right after the Dense(128) layer, just before the final output layer. Looking at my training curves, validation accuracy actually tracked very close to (and for a good stretch, higher

than) training accuracy throughout training, and validation loss ended lower than training loss (0.2196 vs 0.2811 near the best epoch). That's a strong sign the combination of dropout and data augmentation kept the model from just memorizing the training images — without them, I'd expect training accuracy to keep climbing well past validation accuracy, with the validation loss curve turning upward while training loss kept falling.

Q8. Early stopping

Yes, I used early stopping. I monitored `val_accuracy` with a patience of 5 epochs, and set `restore_best_weights=True` so the model rolls back to whichever epoch actually had the best validation accuracy rather than keeping the weights from the very last epoch. I allowed a maximum of 30 epochs, and training stopped automatically after 28 — the best validation accuracy (91.93%) actually happened at epoch 23, and the model restored those weights once it saw 5 epochs pass without further improvement.

Section C: Evaluation and Interpretation

Q9. Test accuracy, confusion matrix, and weakest class

My model scored 90.85% accuracy on the test set, with a test loss of 0.2448. Looking at the confusion matrix, the Shirt class had the lowest accuracy by a clear margin, at around 75.5%, while everything else stayed above 83%. The most likely reason is visual similarity — at a tiny 28x28 grayscale resolution, Shirts look a lot like T-shirts/tops, Pullovers, and Coats (all upper-body garments with a similar silhouette). That shows up directly in the matrix: 100 T-shirt/top images were predicted as Shirt, 84 Coat images were predicted as Shirt, and 65 Pullover images were predicted as Shirt — so the model is genuinely struggling to tell these similar-looking upper-body garments apart rather than making random mistakes.

Q10. Overfitting check

From my training vs. validation curves, training accuracy climbed steadily up to about 90% by the last epoch, while validation accuracy stayed at or above that the whole time, ending around 91.6%. Validation loss also stayed lower than training loss for most of training. That tells me the model isn't overfitting in the classic sense — I don't see training accuracy pulling far ahead of validation accuracy or validation loss rising while training loss falls. I'd credit that mainly to the combination of dropout (which forces the dense layer not to rely too heavily on any single set of neurons) and data augmentation (which makes every training epoch a bit harder by feeding in altered versions of the images), which together closed — and even inverted — the usual train/validation gap.

Section D: Extension and Reflection

Q11. One architectural change with more compute

If I had more time and compute, I'd add Batch Normalization after each convolutional layer, before the activation function. It normalizes the outputs of each layer as training goes, which tends to stabilize and speed up training, lets you safely use a slightly higher learning rate, and often squeezes out a bit more accuracy on top of what dropout alone gives you — especially useful if I also decided to make the network deeper.

Q12. Deployment considerations for a mobile app

For deploying this on a mobile app, I'd first note that the model is already fairly light — about 241K parameters and under 1MB — which is a good starting point for on-device use. Even so, I'd apply post-training quantization (converting it to TensorFlow Lite with int8 weights) to shrink it further and speed up inference on phone hardware. I'd also want to actually measure latency on a real device rather than assume it from parameter count, and test the model against real photos taken with a phone camera — different lighting, backgrounds, and angles than the clean, centered Fashion-MNIST images — since that mismatch (domain gap) is often where a model that looks great on paper starts making mistakes in the real world.